DATA 612 · Statistical Programming in R · Week 1

Lab 1: Looking Under the Hood of R

Predict, then run. The point is to catch R in the act.

Work in an **R script**, not only in the console. Save it as `lab01.R`.

Do **1.1**, **1.2**, **1.3**, and **1.4** first. You will need `tidyverse` installed as in note 1.2, and you will import `students.csv` as in note 1.3. Note **1.5** is about R Markdown and Quarto; you do not need it for this lab.

Download students.csv and put it in your working directory (`getwd()` should see it).

For every item marked **Predict**, do the following in order:

1. Write your prediction in a comment, *before* you run the code.
2. Run the code.
3. Write a short explanation of *why* R did that. "R printed that" is not an explanation.

Some of these questions are meant to surprise people who already use R. That is the point. The notes for 1.2, 1.3, and 1.4 are enough to *start* each part; they are not enough to finish it.

Do not look up the answer until you have a written prediction.

A. What `c()` actually builds

R's atomic vectors are not "lists of values of any type." They are typed objects. `c()` is willing to rewrite your values in order to keep that true.

A1. Predict

```
typeof(c(1, TRUE))
typeof(c(1L, TRUE))
typeof(c(1, "2", TRUE))
```

Why are the three answers not the same? What did `TRUE` become in each case?

A2. Predict

```
TRUE + TRUE + TRUE
c(TRUE, FALSE, TRUE) * 2
```

If `TRUE` is a logical value, why is arithmetic legal?

A3. Predict

```
1 == 1L
identical(1, 1L)
typeof(1)
typeof(1L)
is.numeric(1L)
```

`==` and `identical()` are answering different questions. Which question is each one answering?

A4. Predict

```
f <- factor(c("10", "2", "2"))
as.numeric(f)
as.numeric(as.character(f))
```

This is one of the most expensive mistakes in R. What is `as.numeric()` looking at when the object is a factor?

A5. Predict

```
is.vector(c(1, 2, 3))
is.vector(list(1, "a"))
is.atomic(list(1, "a"))
is.vector(matrix(1:4, nrow = 2))
is.vector(factor(c("a", "b")))
```

If you believed "vector" means "atomic vector," this part exists to break that belief. What does `is.vector()` actually test?

B. Three kinds of nothing

`NA`, `NaN`, and `NULL` are not three names for the same idea. R will not let you treat them as if they were.

B1. Predict

```
x <- c(NA, NaN, Inf, NULL)
x
length(x)
typeof(x)
```

You passed four things to `c()`. Why does `x` not have length 4?

B2. Predict

```
NA == NA
NaN == NaN
NULL == NULL
```

Then:

```
identical(NA, NA)
identical(NaN, NaN)
identical(NULL, NULL)
```

Why can `==` refuse to say that a missing value equals a missing value? Why is `NaN == NaN` different from `identical(NaN, NaN)`?

B3. Predict

```
is.na(NaN)
is.nan(NA)
typeof(NA)
typeof(c(NA, 1))
typeof(c(NA, "a"))
```

`NA` looks like one object. It is not. What happened to `NA` when it entered those two vectors?

B4. Predict

```
mean(numeric(0))
sum(numeric(0))
mean(NULL)
sum(NULL)
```

Empty is not the same as missing, and missing is not the same as absent. Which of these is `NaN`, which is `0`, and which is a warning?

B5. Predict

```
c(1, 2)[0]
c(1, 2)[3]
c(1, 2)[NA]
```

Out-of-range indexing in R does not behave like Python. What object do you get in each case, and why is `[0]` not an error?

C. Indexing is a language

`[` is not "get the nth thing." It is a function with rules for integers, negatives, logicals, names, and recycling.

C1. Predict

```
x <- 10:13
x[c(1, 1, 1)]
x[-c(1, 1)]
```

Positive indices select. Negative indices drop. Why does repeating `1` mean two different things in these two lines?

C2. Predict

```
x <- c(10, 20, 30, 40)
x[c(TRUE, FALSE)]
x[c(TRUE, FALSE, TRUE)]
```

The first logical index is shorter than `x`. The second is not the same length either. What rule is R using, and where does the extra `TRUE` go?

C3. Predict

```
student <- list(name = "Ada", age = 36, grades = c(90, 85))
student[1]
student[[1]]
student$n
student[["n"]]
student["n"]
```

`$` is doing something that `[[` refuses to do. What is it, and why is that dangerous?

C4. Predict

```
students <- data.frame(name = c("Ada", "Bob"), grade = c(91, 74))
class(students[, "grade"])
class(students[, "grade", drop = FALSE])
class(students["grade"])
class(students[["grade"]])
```

One of these is a data frame, one is a vector, and the difference is easy to miss in a larger script. Which is which, and what is `drop` for?

C5. Predict

```
m <- matrix(1:6, nrow = 2)
m
m[2, 1]
```

Fill in the matrix on paper before you run this. R fills matrices in a particular order. Which order, and why is `m[2, 1]` equal to 2 rather than 4?

C6. Predict

```
c(list(1), 2)
list(1, 2)
c(list(1), c(2, 3))
list(1, c(2, 3))
```

`c()` concatenates. `list()` nests. Count the top-level elements in each result, and explain why the last two are different shapes.

D. Packages, names, and what R is searching

A function name is not a unique identity. It is a name R looks up on a search path.

D1. Explain, then try only the quoted form for installation

Why does this fail (do not keep running it):

```
install.packages(ggplot2)
```

while this is the intended form:

```
install.packages("ggplot2")
```

and yet both of these can succeed:

```
library(ggplot2)
library("ggplot2")
```

What is `library()` doing with an unquoted name that `install.packages()` will not do?

D2. Predict, in a session where you have not yet loaded dplyr

```
students <- data.frame(name = c("Ada", "Bob"), grade = c(91, 74))
filter
filter(students, grade > 80)
```

Then load dplyr and run the same two lines again.

```
library(dplyr)
filter
filter(students, grade > 80)
```

There is already a `filter()` in Base R / stats. What changed after `library(dplyr)`? How can two functions share a name? How do you call the older one anyway?

D3. Predict

```
x <- c(10, 20, 30)
x |> mean()
x %>% mean()
```

Run this *before* loading tidyverse. Why does one pipe exist in Base R and the other does not? After `library(tidyverse)`, try:

```
mtcars |> lm(mpg ~ wt, data = _)
```

What is `_` doing, and why is it necessary here when it was not necessary in `x |> mean()`?

D4. Predict

```
search()
library(ggplot2)
search()
```

Where did `ggplot2` go on the search path, and why does that position matter when two packages contain the same function name?

E. Arithmetic that is not the arithmetic you learned in school

E1. Predict

```
c(1, 2) + c(10, 20, 30, 40)
1:3 * 1:6
```

Write the recycled vectors out by hand. When does R warn, and when does it stay silent?

E2. Predict

```
0.1 + 0.2 == 0.3
sqrt(2)^2 == 2
identical(0.1 + 0.2, 0.3)
```

R is not "bad at arithmetic." What class of numbers is it actually using, and why is `==` the wrong tool here?

E3. Predict

```
round(1.5)
round(2.5)
round(3.5)
round(4.5)
```

If you expected "halves round away from zero" or "halves round up," you will get this wrong. What rule is `round()` using, and why would a language choose it?

E4. Predict

```
1 / 0
0 / 0
TRUE / FALSE
Inf - Inf
NA + 1
NULL + 1
```

Which of these are `Inf`, `NaN`, `NA`, and which is an error? What coercion makes `TRUE / FALSE` legal?

F. Files, data frames, and silent shape changes

Place `students.csv` in your working directory.

F1. Run and compare

```
base_students <- read.csv("students.csv")
tidy_students <- readr::read_csv("students.csv")

class(base_students)
class(tidy_students)
str(base_students)
str(tidy_students)
```

Both read the same file. Why are the objects not the same class? What extra classes does `read_csv()` attach, and what is a tibble claiming to be besides a data frame?

F2. Predict

```
base_students$grade * 0.01
base_students[, "grade"]
base_students["grade"]
base_students$g
```

One of these uses the vectorized arithmetic from note 1.4. One may use partial matching. Which extraction still has two dimensions?

F3. Predict

```
T <- 0
T
TRUE
if (T) "yes" else "no"
```

`TRUE` is not the same kind of name as `T`. What is `T` in a fresh session, and why is assigning to it legal when assigning to `TRUE` is not?

G. Optional: the questions that are meant to bother you

Do these last. They are not extra credit in the sense of being optional forever; they are the questions the rest of the lab is training you to ask.

G1. Predict

```
any(c(FALSE, NA))
all(c(TRUE, NA))
any(c(TRUE, NA))
all(c(FALSE, NA))
```

`NA` means "unknown." Translate each line into English with the word *unknown* in it, then check whether R agrees with your English.

G2. Predict

```
unique(c(NaN, NaN))
duplicated(c(NA, NA))
duplicated(c(NaN, NaN))
```

If `NaN == NaN` is `FALSE`, how can `unique()` collapse two `NaN`s? What notion of sameness is `unique()` using?

G3. Predict

```
1:0
1:(-1)
seq_len(0)
seq_along(NULL)
```

`:` is not "the empty range" when the ends look empty. What does `1:0` actually generate, and when would `seq_len()` be the safer function?

G4. Predict

```
x <- 1:5
y <- x
x[1] <- 99L
x
y
```

You might think `y` is an alias for `x`. In R it usually is not, after you modify `x`. What did the assignment `y <- x` actually copy, and when did R bother to copy it?

G5. Predict

```
gender <- factor(c("Male", "Female", "Female"))
c(gender, "Other")
```

You asked for another category. What values do you actually get, and what did `c()` do to the factor before it could append a character?

What to submit

Submit `lab01.R` with:

- a prediction comment above every **Predict** block
- the code
- a short explanation comment below the output of each block

If a result still feels impossible after you have explained it, write down the question you now have. That question is part of the lab.